

DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 10

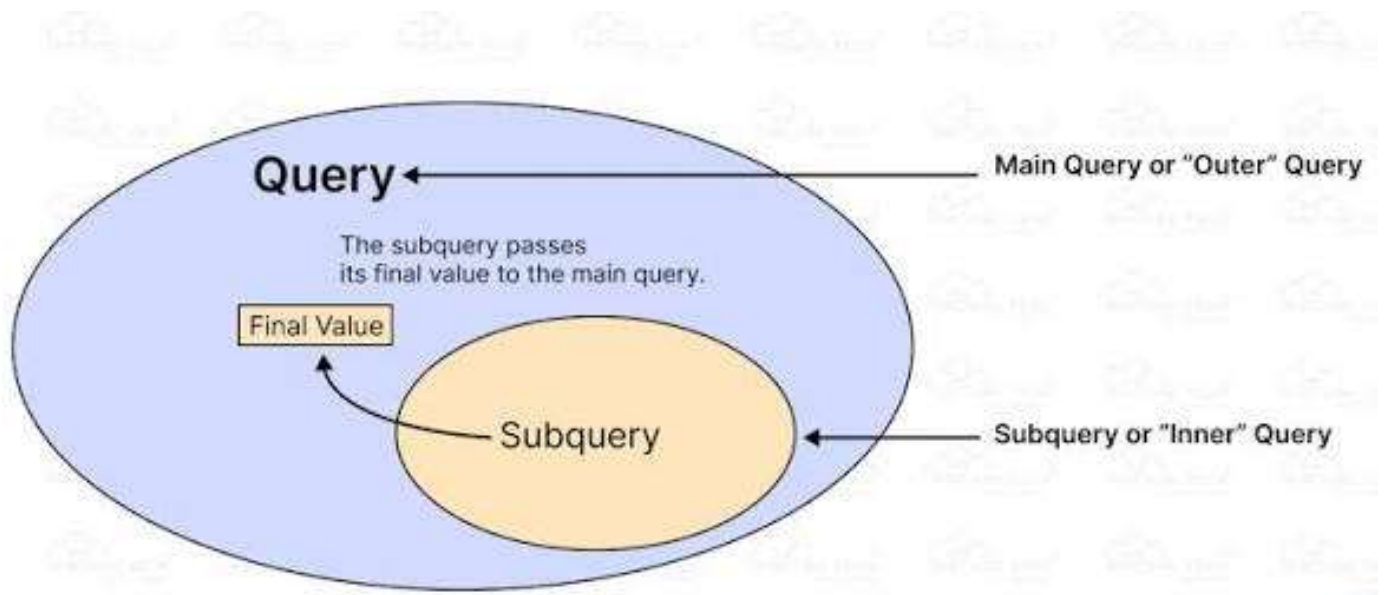
Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

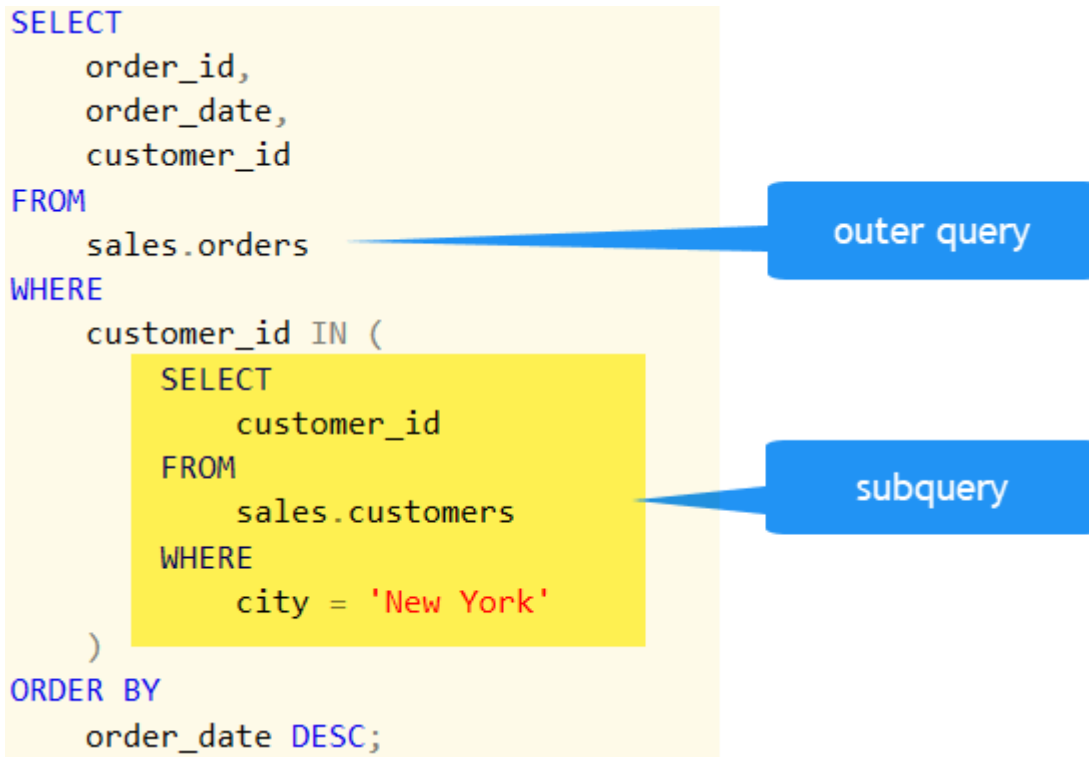
1. Subqueries and Nested Queries

A **subquery** is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.



Basic Structure



Types of Subqueries

Type	Description
Single-row subquery	Returns one value
Multiple-row subquery	Returns multiple values
Nested subquery	A subquery inside another subquery
Correlated subquery	Inner query depends on the outer query

Example

Find orders whose amount is greater than the average order amount.

```

SQL> SELECT Order_ID, Customer_ID, Total_Amount
2 FROM Orders
3 WHERE Total_Amount >
4 (SELECT AVG(Total_Amount)
5 FROM Orders);
  
```

ORDER_ID	CUSTOMER_ID	TOTAL_AMOUNT
201	101	2700
203	103	3200

Explanation

The inner query:

```
SELECT AVG(Total_Amount) FROM Orders;
```

calculates the average order amount.

The outer query then displays orders whose Total_Amount is greater than that average.

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price.

Assuming the Product table contains:

- Product_ID
- Product_Name
- Price

Query

```
SQL> SELECT
2     Product_ID,
3     Product_Name,
4     Price
5 FROM Product
6 WHERE Price >
7       (SELECT AVG(Price)
8        FROM Product);
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
```

```
PRICE
-----
      103
Organic Honey
      350

      104
Bamboo Storage Box
      450
```

Explanation

The inner query calculates the average price:

```
SELECT AVG(Price) FROM Product;
```

The outer query displays products whose price is greater than the calculated average.

The actual output depends on the Product data in the database.

2.1 Products Below Average Price

The same concept can be used to find products below the average price.

```
SQL> SELECT
  2     Product_ID,
  3     Product_Name,
  4     Price
  5 FROM Product
  6 WHERE Price <
  7     (SELECT AVG(Price)
  8      FROM Product);
```

PRODUCT_ID	PRODUCT_NAME	PRICE
101	Organic Turmeric Powder	130
102	Black Pepper	180

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT
  2     Customer_ID,
  3     SUM(Total_Amount) AS Total_Purchase
  4 FROM Orders
  5 GROUP BY Customer_ID;
```

CUSTOMER_ID	TOTAL_PURCHASE
102	1800
101	2700
104	1500
103	3200

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(Total_Purchase)
2 FROM
3 (
4     SELECT
5         Customer_ID,
6         SUM(Total_Amount) AS Total_Purchase
7     FROM Orders
8     GROUP BY Customer_ID
9 );
```

MAX(TOTAL_PURCHASE)
3200

The inner query calculates the total purchase for each customer.

The outer query finds the maximum value among those totals.

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT
2     c.Customer_ID,
3     c.Customer_Name,
4     SUM(o.Total_Amount) AS Total_Purchase
5 FROM Customer c
6 JOIN Orders o
7     ON c.Customer_ID = o.Customer_ID
8 GROUP BY
9     c.Customer_ID,
10    c.Customer_Name
11 HAVING SUM(o.Total_Amount) =(
12     SELECT MAX(Total_Purchase)
13     FROM (
14         SELECT SUM(Total_Amount) AS Total_Purchase
15         FROM Orders
16         GROUP BY Customer_ID
17     ));
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_PURCHASE
103	Priya	3200

Explanation

The innermost query:

```
SELECT SUM(Total_Amount)
```

```
FROM Orders
```

```
GROUP BY Customer_ID;
```

calculates the purchase amount for every customer.

The next query:

```
SELECT MAX(Total_Purchase)
```

finds the highest purchase amount.

The main query displays the customer whose purchase amount matches that maximum.

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT
 2      c.Customer_ID,
 3      c.Customer_Name,
 4      o.Order_ID,
 5      o.Total_Amount
 6 FROM Customer c
 7 JOIN Orders o
 8      ON c.Customer_ID = o.Customer_ID
 9 WHERE o.Total_Amount >
10 (
11      SELECT AVG(Total_Amount)
12      FROM Orders
13 )
14 ORDER BY o.Total_Amount DESC;
```

CUSTOMER_ID

CUSTOMER_NAME

	ORDER_ID	TOTAL_AMOUNT
Priya	103	
	203	3200
Anitha Kumar	101	
	201	2700

Purpose

This report identifies customers who placed orders whose value is greater than the overall average order value.

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Details.

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     SUM(od.Subtotal) AS Total_Sales
  5 FROM Product p
  6 JOIN Order_Details od
  7     ON p.Product_ID = od.Product_ID
  8 GROUP BY
  9     p.Product_ID,
 10     p.Product_Name
 11 HAVING SUM(od.Subtotal) >(
 12     SELECT AVG(Product_Sales)
 13     FROM(
 14         SELECT SUM(Subtotal) AS Product_Sales
 15         FROM Order_Details
 16         GROUP BY Product_ID
 17     ))
 18 ORDER BY Total_Sales DESC;
```

PRODUCT_ID

PRODUCT_NAME

TOTAL_SALES

101	
Organic Turmeric Powder	
4500	
104	

Purpose

This query identifies products whose total sales are greater than the average sales generated by the products.

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT
2     c.Customer_ID,
3     c.Customer_Name,
4     COUNT(o.Order_ID) AS Total_Orders,
5     SUM(o.Total_Amount) AS Total_Purchase,
6     AVG(o.Total_Amount) AS Average_Order_Value
7 FROM Customer c
8 JOIN Orders o
9     ON c.Customer_ID = o.Customer_ID
10 GROUP BY
11     c.Customer_ID,
12     c.Customer_Name
13 ORDER BY Total_Purchase DESC;
```

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE

103			
Priya	1	3200	3200
101			
Anitha Kumar	1	2700	2700

Report Contains

- Customer ID
- Customer Name
- Number of orders
- Total purchase amount
- Average order value

7. Advanced Product Sales Report

This report combines product and order details.

```
SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     COUNT(od.Order_ID) AS Number_of_Orders,
5     SUM(od.Quantity) AS Total_Quantity_Sold,
6     SUM(od.Subtotal) AS Total_Sales,
7     AVG(od.Unit_Price) AS Average_Price
8 FROM Product p
9 JOIN Order_Details od
10    ON p.Product_ID = od.Product_ID
11 GROUP BY
12     p.Product_ID,
13     p.Product_Name
14 ORDER BY Total_Sales DESC;
```

PRODUCT_ID	PRODUCT_NAME	NUMBER_OF_ORDERS	TOTAL_QUANTITY_SOLD	TOTAL_SALES	AVERAGE_PRICE
101	Organic Turmeric Powder	2	4	4500	1250
104	Bamboo Storage Box	1	2	3200	1600

Purpose

This report helps the business identify:

- Number of orders for each product
- Total quantity sold
- Total sales
- Average selling price
- Products generating higher sales

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Details.

```
SQL> SELECT
2     cat.Category_ID,
3     cat.Category_Name,
4     COUNT(DISTINCT od.Order_ID) AS Total_Orders,
5     SUM(od.Quantity) AS Total_Quantity_Sold,
6     SUM(od.Subtotal) AS Total_Sales,
7     AVG(od.Unit_Price) AS Average_Price
8 FROM Category cat
9 JOIN Product p
10    ON cat.Category_ID = p.Category_ID
11 JOIN Order_Details od
12    ON p.Product_ID = od.Product_ID
13 GROUP BY
14     cat.Category_ID,
15     cat.Category_Name
16 ORDER BY Total_Sales DESC;
```

CATEGORY_ID	CATEGORY_NAME	TOTAL_ORDERS	TOTAL_QUANTITY_SOLD	TOTAL_SALES	AVERAGE_PRICE
1	Spices	5	5000	1000	
3	Eco Friendly Home	2	3200	1600	